

Supplementary Material for From Prompt to Policy: Controlling LLM Transformations in Healthcare Communication

This supplement documents the bounded verification artifact `SAI-CARE-control`. It provides the enumeration domain, observed results, fixture execution details, reproduction instructions, and verification limits. The complete conformance cases C1–C23 are listed in the main paper. Rule identifiers R1–R11 and invariant identifiers I1–I8 refer to the definitions in that paper. The artifact implements a finite policy abstraction and selected control mechanisms using scripted assessments and in-memory services; it is not a deployed healthcare application.

APPENDIX A BOUNDED EXECUTABLE VERIFICATION

A. Artifact and Evaluation Scope

The artifact comprises a pure policy evaluator, an exhaustive enumerator over a finite input domain, a synchronous in-memory control prototype, and twenty-three scripted fixture groups. The evaluator returns a terminal rule, decision mode, resolution destination, and obligations. The control prototype represents input compilation, origin-aware validation, review obligations, version-bound approval, and release checks. Model outputs, assessment labels, and semantic verdicts are scripted.

The enumeration examines policy decisions for supplied structured attributes. The fixtures additionally observe selected processing calls, compiled payloads, held candidates, attribution records, approval bindings, and dispatch records. These are complementary checks: enumeration covers the defined abstract domain, whereas fixtures exercise representative execution paths. Neither establishes universal conformance of a production implementation.

B. Enumeration Domain

The raw Cartesian product contains four actors, four operations, two request phases, eleven task labels, four risk labels, two classification statuses, and four Boolean dimensions:

- Actors: owner, caregiver, model, and healthcare professional.
- Operations: transform, edit, approve, and share.
- Phases: initiate and execute.
- Tasks: TR-1 through TR-9, unknown, and not applicable (NA).
- Risks: low, moderate, high, and unassessed.
- Classification status: resolved or uncertain.
- Boolean dimensions: explicit scope prohibition, resolved scope, satisfied preconditions, and a configured professional route.

This gives $4 \times 4 \times 2 \times 11 \times 4 \times 2 \times 2^4 = 45,056$ raw tuples. The following consistency constraints retain 8,848 well-formed states:

- 1) The task is NA if and only if the operation is not transform.
- 2) An unknown task implies uncertain classification.
- 3) A transformation with unassessed risk has uncertain classification.
- 4) Non-generative operations use resolved classification by convention.
- 5) Owner requests use the execute phase.

The unknown task label implements $\perp$ in the main specification. Permission and readiness predicates are abstract inputs; concrete documents, identity records, delegation credentials, consent histories, and service implementations are not enumerated. Scope prohibition and scope resolution may both be true, because a resolved permission can be a denial. The route dimension is retained even where it does not affect the decision. Consequently, rule counts describe this enumeration, not the frequency of real requests.

C. Checks and Observed Results

For each well-formed state, the evaluator is compared with the first satisfied predicate in the ordered rule specification. Repeated evaluation checks consistency on identical inputs. Additional assertions check the conditional high-risk routing boundary, the actor and phase reaching R9, and the operations for which a caregiver reaches R10. For transformation states with low or moderate risk, the enumerator also replaces the risk with high while keeping other attributes fixed and checks that R1 takes precedence.

All 8,848 states return a decision. The checks report zero first-match discrepancies, zero repeat-evaluation discrepancies, and zero routing violations of I2. Only owner/execute states reach R9, and caregiver states reaching R10 concern editing. All 4,256 low/moderate-to-high risk-strengthening pairs select R1. Table I reports the terminal-rule counts.

TABLE I
TERMINAL-RULE REACHABILITY IN THE FINITE ABSTRACTION.

Terminal rule	Number of states
R1	3,248
R2	4,000
R3	800
R4	400
R5	192
R6	88
R7	8
R8	56
R9	24
R10	32
R11	0
Total	8,848

Ten terminal rules are reachable. R11 is a defensive fallback: after the preceding checks, R9 and R10 exhaust the remaining well-formed operations. Invalid states are rejected at intake and are not treated as R11 cases.

These results concern the supplied attributes. In particular, the enumerated I2 check establishes routing to R1 under its stated condition; selected fixtures separately observe the absence of transformation calls. Repeat evaluation is a regression check, not an independent proof of determinism or correctness. The ordered-predicate comparison shares the policy specification with the evaluator and does not independently validate the requirements themselves.

D. Control-Path Fixtures

All twenty-three fixture groups pass their assertions. A group can contain multiple subcases, so this count is neither a count of independent samples nor a statistical performance estimate. The fixtures observe rules, processing calls, payloads, validation outcomes, attribution, approval bindings, and dispatch records.

Version, scope, consent, and provenance changes are injected synchronously before the simulated dispatch boundary. The delivery stub uses a stable identifier to exercise reconciliation without duplicate dispatch. The audit stub can be made unavailable before an effect. These mechanisms represent selected failure conditions without implementing real concurrency, persistent journalling, or network transport.

Version 4 extends C2 to both professional-route outcomes and checks the fixed, deployment-owned urgent-care message. C5 and C16 exercise recipient-visible provenance finalisation and digest invalidation. C21 exercises exclusions across sources, parameters, preferences, and metadata, including blocking task-changing exclusions until reconfirmation. C19 supplies the injection-detection verdict: it checks containment after detection, not the ability of a validator to discover an attack.

APPENDIX B

REPRODUCTION INSTRUCTIONS AND VERIFICATION LIMITS

A. Package and Execution

The artifact identifier is SAI-CARE-control. The accompanying source package contains the following files:

- `verification/policy.py`: state and decision records, consistency constraints, ordered policy rules, and the enumeration generator.
- `verification/control.py`: the synchronous in-memory control prototype.
- `verification/verify.py`: enumeration checks and the twenty-three fixture groups.
- `verification/results.json`: machine-readable results generated by the verification script.
- `verification/README.txt`: domain definitions, interpretation, and verification scope.

Python 3.10 or later and its standard library are sufficient. No network connection, model API, patient data, or external Python package is required. From the source-package directory, run:

```
python3 verification/verify.py
```

The command exits with a nonzero status if an assertion fails and rewrites `verification/results.json`. The expected output contains 8,848 well-formed states, the rule counts in Table I, zero violations for the enumerated checks, 4,256 risk-strengthening pairs, and a PASS result for each of C1–C23. The source package includes a checksum manifest for identifying the supplied files. No public repository or archival deposit is assumed by these instructions.

B. Coverage of the Invariants

The artifact exercises the following aspects of the main paper’s invariants. Passing these checks does not establish the invariants over all possible execution traces.

I1: Exact authorised disclosure. C1, C5, C6, C8, C12, C14, C16, and C20 exercise representative authority, approval-binding, and dispatch paths. Identity services, recipient authentication, and distributed release transactions are not implemented.

I2: Conditional generative boundary. The routing implication is checked throughout the finite state domain; C2 and C11 additionally observe the absence of transformation calls. Correct detection of task and risk is outside this verification.

I3: Origin-aware validation. C1, C7, C9, C18, C20, and C22 exercise scripted origin, lineage, flag, attribution, and review gates. The truth of statements and semantic faithfulness of real model outputs are not assessed.

I4: Exact whole-input authorisation. C3, C15, C17, C21, and C23 exercise service gates and the represented compiler channels. Parameters, preferences, and metadata have explicit item handles subject to authorisation and exclusion. A complete derivation graph and production middleware are not implemented.

I5: Explicit personalisation. C10 checks omission of a disabled preference. Preference persistence, deletion, and interface behaviour are not implemented.

I6: No implicit release path. Selected fixtures check that initiation, blocked transformations, validation holds, and execution failures do not dispatch data. C19 supplies the detection verdict rather than testing a detector.

I7: Audit completeness for effects. C13 and C16 exercise unavailable-journal and dispatch-record stubs. Persistent durability, crash recovery, and third-party transport acknowledgement are not tested.

I8: No implicit consent. C4, C17, and C21 exercise pauses and explicit narrowing without inferred consent. Scheduled expiry, real timeout handling, and a production consent workflow are not implemented.

C. Interpretation and Remaining Limits

The prototype treats structured policy attributes and service verdicts as supplied inputs. Semantic flags, reviewer evidence, and model responses are scripted; a successful fixture does not show that a real classifier, validator, or reviewer would produce the correct result. The fixed urgent-care message is synthetic deployment configuration used to test routing and content isolation, not clinically evaluated guidance.

Approval finalisation is represented as a synchronous operation that freezes recipient-visible labels, computes the bundle digest, and stores a detached approval record. Mutation hooks exercise changes between stages, but do not establish atomicity under concurrent requests. Likewise, the in-memory journal and delivery map cannot demonstrate persistent audit completeness, recovery after partial failures, or interoperability with an external delivery service.

The artifact therefore supports inspection and reproduction of selected properties of the specified control model. Clinical safety, adversarial robustness, usability, legal compliance, and production readiness require additional implementation and empirical evaluation.